

CASCO INTELIGENTE CON IA PARA DETECCIÓN Y RESPUESTA A ACCIDENTES DE MOTOCICLISTAS

M C N

El **Casco Inteligente con IA** es un sistema de seguridad activa diseñado para motociclistas, capaz de **detectar accidentes en tiempo real** mediante la combinación de múltiples sensores y un modelo de inteligencia artificial entrenado para diferenciar eventos críticos de situaciones normales.

Ante un accidente confirmado, el casco ejecuta **acciones automáticas** como **GPS**, grabar y almacenar el video previo al impacto, activar luces de emergencia y notificar a contactos de emergencia y/o servicios médicos.

2) Mejor de los casos

- **Detección fiable** de un impacto fuerte o una caída en $\leq 2-3$ s desde el evento.
- **Cuenta regresiva configurable** (10-20 s): si el piloto responde "Estoy bien", se cancela; si no, se envía **SOS** con **GPS** preciso ($\pm 5-10$ m) y **historial de 10-20 s** de telemetría.
- **Falsos positivos bajos** ($\leq 2\%$ por hora de conducción) y **falsos negativos muy bajos** en impactos reales ($\geq 95\%$ de sensibilidad en pruebas controladas).
- **Batería** del módulo ≥ 8 h de uso continuo (escaneo adaptativo y bajo consumo cuando está detenido).
- **Privacidad****: datos en el teléfono; subida opcional a nube; cifrado de backups.
- **Auxiliar**: app Flutter con panel, historial, ajustes y protocolo claro App $\rightleftharpoons$ ESP32.

3) Alcance y casos de uso

Rangos de velocidad (para calibración de umbrales/modelo):

- **Urbano**: 0-50 km/h
- **Carretera**: 50-100 km/h
- **Autopista**: 100-150 km/h

Casos de uso principales:

1. **Detección de impacto/caída** → alerta + countdown → SOS opcional.
2. **Riesgo alto** (combinación de inclinación + velocidad + desaceleración) → alerta preventiva (vibro/háptico o audio breve).
3. **Somnolencia/fatiga** (heurística básica con patrones de microoscilación + tiempo de conducción) → sugerencia de descanso (sin SOS).

4. **Registro de evidencia** (si hay cámara/micrófono): buffer circular local + “congelar” 10 s antes y 10 s después del evento.

Fuera de alcance v0:

- Reconocimiento de señales de tránsito.
- Navegación/Mapas integrados (se añade en etapa 2).
- Diagnóstico de colisiones complejas de múltiples vehículos.

4) Variables, sensores y parámetros

IMU (Unidad de Medición Inercial)

- Acelerómetro (ax, ay, az) [g] – muestreo 100–200 Hz.
- Giroscopio (gx, gy, gz) [°/s] – muestreo 100–200 Hz.
- Inclinación/roll-pitch-yaw estimados por fusión (Madgwick/Complementario) .

Batería

- Nivel de porcentaje, tensión, temperatura.

Eventos derivados

- **Jerk** (derivada de aceleración) [g/s].
- **Desaceleración pico** ($-a_{max}$) en $\Delta t = 100\text{--}300\text{ ms}$.
- **Ángulo de inclinación** vs **velocidad** (curvas a alta velocidad).
- **Vibración RMS** (ventanas de 1–2 s) como proxy de firmeza del pavimento.
- **Clasificador de movimiento**: detenido / marcha / frenada / curva.

Umbrales iniciales (regulación de seguridad)

- Impacto “duro”: $|a| > 2.5\text{--}3.0\text{ g}$ sostenido $> 50\text{ ms}$ + cambio brusco de orientación ($> 25\text{--}40^\circ$ en $< 200\text{ ms}$).
- Desaceleración severa: $> -6\text{ m/s}^2$ (contextualizado por velocidad).
- Curva riesgosa a alta velocidad: inclinación $> 35^\circ$ a $> 100\text{ km/h}$.
- **Regla de oro**: a mayores velocidades, reducir tolerancias (bandas adaptativas por rango).

Nota: Estos son **valores de arranque** para pruebas controladas; se recalibran con datos reales.

5) Arquitectura técnica

Hardware (módulo en casco)

- **ESP32** (microcontrolador Wi-Fi/BLE): adquisición de IMU/GPS, preprocesado y eventos.
- **IMU** (MPU-6050/MPU-9250/ICM-20948), **GPS (u-blox/NEO-6/ESP32 GNSS)**, **Batería** + BMS.
- Opcional: **ESP32-CAM** + **microSD** (buffer circular).

Software en el casco

- Muestreo → Filtro (Madgwick) → Extracción de features → Heurísticas + **modelo IA ligero** (si entra en ESP32) → Eventos JSON vía BLE/Wi-Fi.

App Flutter

- Recepción de telemetría/eventos (BLE GATT o Wi-Fi WebSocket/HTTP).
- Alerta con countdown; **ACK (Acknowledgement = confirmación)** al casco.
- Historial local; envío de **SMS/Push** a contactos (opcional nube).

Protocolo JSON (simplificado)

- Telemetría: {t, imu:{ax,ay,az,gx,gy,gz}, gps:{lat,lon,spd,hdop}, bat, flags}
- Evento: {type:'impact' | 'risk' | 'drowsy', utc, lat, lon, confidence}
- ACK App→Casco: {cmd:'ack', ok:true | false, wait:segundos}

6) IA y “regulación” (cómo se controla)

Entrenamiento offline

- Dataset etiquetado por rango de velocidad (urbano / carretera / autopista).
- Etiquetas: normal, frenada brusca, curva riesgosa, impacto/caída.
- Balance de clases; validación cruzada; métricas: **recall** prioritario para impacto (FN bajos), **precision** aceptable para minimizar falsas alertas.

Inferencia en tiempo real

- **Capas de seguridad:**
 1. Heurísticas/umbrales duros (hard limits por física y velocidad).
 2. IA supervisada (clasificador ligero: Random Forest / SVM / Micro NN).
 3. Fusión de decisiones: se alerta si (heurística **o** IA) > umbral de confianza; se confirma con persistencia temporal para evitar picos.
- **Fail-safe:** si IA no está disponible, se mantiene heurística con umbrales conservadores.

7) Limitaciones y supuestos

- **Ruido de IMU y deriva:** requiere calibración y fusión de sensores.
- **GPS con pérdida de señal** (túneles/edificios): degradar a estimación inercial + última posición válida.
- **Variabilidad de cascos/motos/usuarios:** cambios en vibración y postura → necesidad de recalibración por usuario (perfil rápido al inicio).
- **Batería:** trade-off entre tasa de muestreo y autonomía.
- **Clima:** lluvia/viento pueden alterar patrones; IA debe ver ejemplos.

8) Riesgos y mitigaciones

Riesgo	Impacto	Prob.	Mitigación
Falsos positivos (alertas molestas)	Medio	Media	Histeresis temporal, consenso (heurística+IA), umbrales por rango.
Falsos negativos (no detectar impacto)	Alto	Baja/Media	Priorizar recall , sensores bien montados, pruebas de caída controladas.
Pérdida de GPS	Medio	Media	Última posición válida + reconexión + BLE a teléfono para posicionamiento alternativo.
Sobrecarga en ESP32	Medio	Baja	Preprocesado eficiente, ventanas cortas, usar fixed-point cuando sea posible.
Privacidad	Alto	Baja	Datos locales por defecto; opt-in para nube; cifrado; anonimización en pruebas.
Consumo de batería	Medio	Media	Modo bajo consumo, muestreo adaptativo, sleep cuando esté detenido.
Hardware suelto en casco	Alto	Baja	Diseño mecánico y pegado adecuados;

9) Validación y métricas de éxito

- **Sensibilidad (recall) impacto** $\geq 95\%$ en pruebas controladas.
- **Tasa de falsos positivos** $\leq 2\%$ por hora de conducción.
- **Latencia detección→alerta** ≤ 3 s.
- **Precisión GPS** en SOS: ≤ 10 m (cielo abierto; registrar HDOP para contexto).
- **Autonomía** ≥ 8 h con perfil urbano.
- **Satisfacción de usuario** (encuesta SUS) $\geq 80/100$ en prototipo.

Plan de pruebas

1. Banco: sacudir en eje, impactos con mazo amortiguado, cambios de orientación controlados.
2. Pista segura: frenadas fuertes, curvas, baches (a 20/40/60/80/100/120 km/h).
3. Campo: trayectos reales urbanos/carretera; registrar incidencias.

10) Plan de trabajo APROX y modificable

1

- Documento de alcance (este), riesgos, KPIs, glosario.
- App Flutter demo (UI, countdown, historial). 

2

- Integrar IMU + GPS en ESP32; publicar JSON por BLE/Wi-Fi.
- Calibración inicial; logging a microSD/teléfono.

3

- Heurísticas base y umbrales por rango de velocidad.
- Pruebas de banco + tuning.

4

- Dataset piloto; entrenamiento IA ligera; comparación con heurística.
- Fusión de decisiones; reducción de falsos positivos.

5

- Pruebas en pista; KPIs; mejoras de UX; preparación demo.
- Dossier de seguridad y privacidad; manual rápido.

La parte del GPS puede ser modificada a no ponerla, ya que puede implicar mas complejidad al momento de poder ajustar el mapa con el microcontrolador

